

Natural Language Task Planning for Autonomous Ground Robot Navigation via Large Language Models

William Kojumian

Abstract—Large language models can translate operator instructions into robot plans, but a valid output format does not establish correct task behaviour. This study evaluates typed navigation commands using a single-call gpt-4o-mini planner, a named-location JSON contract, and a ROS2 Nav2 executor for a simulated TurtleBot3 warehouse robot. A researcher-designed dataset contains 100 commands in five categories: direct, spatial, multi-step, conditional, and ambiguous. Three generation trials per command are assessed structurally; semantic assessment combines automatic matching with 27 single-rater judgements. With the revised prompt, automatic semantic success is 96.7%, 95.0%, and 95.0% for the first three categories. Item-level strict scores are 55.0% for conditional commands and 85.0% for ambiguous commands. This descriptive reversal is consistent with unequal representational support: the contract permits clarification but cannot encode an alternative destination after failure. A targeted schema-and-prompt extension improves conditional schema adherence from 55/60 to 60/60 and passes a narrow branch-encoding check in 7/9 selected trials. These structural results support a representation bottleneck, without establishing a general causal law or improved navigation success. Three additional saved-plan replays reached all nine requested goals; semantic grading of the extension remains outstanding.

Index Terms—Large language models, robot task planning, natural language navigation, JSON schema, ROS2, Nav2, evaluation.

I. INTRODUCTION

WAREHOUSE navigation combines repeated routes, named work areas, and changing requests [1]. A graphical map is effective for choosing one destination. Typed language may help specify ordered patrols, responses to inaccessible areas, or requests needing clarification. This potential advantage motivates the interface; usability is not measured here.

The research question is: *how reliably does translation from English to a navigation plan preserve task intent across different linguistic demands?* One successful route does not measure translator reliability, while a high plan score does not establish successful robot execution.

The prototype separates task specification from motion generation. A large language model (LLM) chooses named destinations and simple actions. An executor resolves names to map coordinates and delegates movement to Nav2. This restricted interface makes translation errors inspectable separately from physical behaviour.

Evaluation uses 100 commands divided equally among direct, spatial, multi-step, conditional, and ambiguous categories, labelled L1–L5. These are intended linguistic demands, not a validated difficulty scale. Strict scores fall at L4 and recover at L5. Explaining this pattern requires examining both the output representation and scoring rules.

The original contract permits clarification but cannot specify “if A cannot be reached, navigate to B instead.” Its failure options name behaviours such as retrying or skipping, not alternative destinations. Observed outputs place conditions in explanatory text while executable steps visit both destinations unconditionally.

The contribution is a reproducible command set and scoring pipeline, an analysis separating validity from task correctness, and a targeted fallback extension. This bounded case study motivates auditing the plan representation before attributing planning failures to model capacity; it does not introduce a general planning algorithm.

The scope is typed language, navigation and waiting, one known warehouse map, and simulation. “Inspect” and “check” are interpreted as navigation requests, not as implemented perception or inspection capabilities. There is no voice input, manipulation, hardware validation, or demonstrated industrial deployment readiness.

II. RELATED WORK

A. Language models and executable robot tasks

SayCan grounds language-based skill selection in affordance estimates [2]. Inner Monologue incorporates environment feedback into continued planning [3]. Both already distinguish linguistic plausibility from successful embodied action; the present contribution concerns a particular navigation contract rather than discovering that plausible plans can fail.

Code as Policies generates programs with robot API calls and control structures [4]. ProgPrompt uses available actions, objects, and example programs to condition generation [5]. Such representations can express branching absent from a flat waypoint contract. Here, a smaller output vocabulary trades expressiveness for inspectability; these alternatives are not experimentally compared.

Vemprala *et al.* examine prompts, function libraries, and dialogue across robotics tasks [6]. ReAct interleaves reasoning and environment interaction [7]. Here, omitting feedback and corrective calls keeps the initial plan as the observable unit, but removes opportunities for subsequent repair.

B. Language-guided navigation and the execution layer

LM-Nav combines pretrained language, vision, and action models for navigation using language descriptions of landmarks [8]. NavGPT uses textual descriptions of visual observations, navigation history, and available directions to select actions sequentially [9]. These settings include perceptual grounding or ongoing navigation context. Here, grounding is deliberately simpler: all 20 location names, descriptions, and coordinates are supplied in a known semantic map. The results therefore concern translation within a closed environment vocabulary, not landmark discovery in an unfamiliar scene.

Nav2 supplies the lower-level robotics infrastructure. The Marathon 2 describes a ROS2 navigation system with behaviour-tree orchestration [10]; the later ROS2 navigation survey situates planning, control, and recovery components within that architecture [11]. The executor developed here operates above Nav2 and sends pose goals. It does not replace the path planner or controller. Behaviour trees can express conditional control flow, so the missing fallback is a limitation of this project’s task contract rather than an inherent limitation of Nav2.

SLAM Toolbox provides mapping and localisation functionality [12], but the reported simulation uses a generated occupancy map and ground-truth odometry. This isolates translation from localisation error while limiting evidence about real-world operation.

C. Structured output and the research distinction

Chain-of-thought prompting investigates explicit intermediate reasoning [13]. The present prompts instead request a structured plan with brief explanatory fields; those fields are not executable and do not provide direct access to the model’s internal reasoning. A plausible explanation accompanying an incorrect plan is evidence about the output, not proof of intact comprehension.

Tam *et al.* study the effect of output-format restrictions on model performance [14]. JSONSchemaBench evaluates constrained decoding in terms of efficiency, schema coverage, and output quality [15]. These establish relevant distinctions between format compliance and useful content. The implementation here uses JSON-object mode plus validation after generation, not decoding constrained by the complete JSON Schema. Its schema errors are consequently observable rather than ruled out during generation.

Hallucination research provides broader context for outputs unsupported by their grounding information [16]. Restricting navigation to known names addresses one narrow problem: arbitrary generated coordinates cannot become accepted co-ordinate fields in a conforming plan. Incorrect existing destinations, invented names, or unsupported claims in comments remain possible. Named targets are therefore an interface restriction, not a general guarantee against hallucination or unsafe action.

Table I positions the combination of contract validation, category-based testing, and a targeted fallback intervention.

TABLE I
RELATIONSHIP TO SELECTED PRIOR WORK

Approach	Established focus	Distinction here
SayCan; Inner Monologue	Affordances and feedback	Fixed map; one translation call
Code as Policies; ProgPrompt	Programs and situated actions	Restricted waypoint contract
LM-Nav; NavGPT	Language with navigation observations	Supplied named locations
Tam; JSON-SchemaBench	Format constraints and output quality	Executable fallback semantics

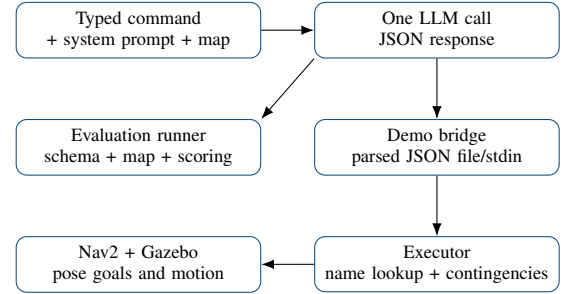


Fig. 1. Implemented evaluation and demo paths. Full schema and map validation belongs to the evaluation runner; the demo bridge does not currently enforce that same gate.

III. SYSTEM DESIGN AND IMPLEMENTATION

A. Two paths through the prototype

The system has a shared translation component and separate evaluation and execution paths (Fig. 1). The operator’s typed command is sent as a user message. The system message combines the robot’s role and capability limits, the semantic map, the output contract, interpretation policies, and worked examples. The planner requests gpt-4o-mini at temperature zero with `response_format=json_object`. There is one application-level generation call per command, with no agent loop or corrective re-prompting.

The evaluation runner parses the response, checks it against the selected JSON Schema, validates navigation targets against the map, and records semantic and contingency scores where applicable. The demo bridge instead writes parsed JSON to a file. The ROS2 executor reads that file or standard input, resolves destination names, and sends goals to Nav2. The current bridge does *not* invoke the full evaluation validator before execution. The report therefore distinguishes the evaluated validation pipeline from the demo’s more limited checks; a universal pre-execution validation gate is not claimed.

B. Map and plan contract

The semantic map describes a 20×20 m warehouse with three aisles and 20 named locations. North is positive y and east is positive x . Each name maps to fixed coordinates and a textual description. The model can use those coordinates when interpreting “nearest” or “midway,” but its navigation output is a name. The executor alone converts that name into a pose. This makes the map the authority for destination coordinates.

TABLE II
EXECUTOR BEHAVIOUR AFTER A NAVIGATION FAILURE (v2)

on_blocked	Executed response
abort	Stop the remaining plan.
skip	Mark the step skipped; continue.
wait_retry	Wait, then retry the same target once; default wait is 5 s.
reroute_perimeter	Visit a route on the perimeter and retry access to the original destination.
null	Record failure; continue to later steps.

At plan level, `understood` indicates whether the model accepts the request. The prompt asks for an empty plan and a useful `clarification_question` when it does not. At step level, `navigate` identifies a target, while `wait` supplies `duration_s`. The `reason` and `notes` fields explain choices but have no control-flow effect. In particular, a condition written in either field does not make a later navigation step conditional.

Table II describes the implemented v2 failure actions. These run after a navigation attempt reports failure. A failed attempt is used as an approximation to “blocked”; there is no separate semantic detector for an occupied dock, noise source, or blocked-aisle count. Nav2 may already have attempted its own recoveries before returning failure.

The JSON Schema checks declared types, enumerations, and disallowed extra properties, but some prompt requirements exist only as descriptions. For example, the v2 schema does not formally enforce every relationship between acceptance, clarification, action, target, and wait duration. A schema pass therefore establishes compliance with the implemented validator, not every intended contract rule. Similarly, map membership does not establish traversability, clearance, or preservation of the operator’s intent.

C. Prompt architecture and revisions

The v2 prompt contains 13 interpretation policies and six worked examples after the contract and map. Policies define traversal versus endpoint navigation, sequence preservation, wall sweeps, perimeter direction, geometric inference, waiting, contingencies, ambiguity, capability limits, and home/base aliases. This is task-specific prompting with examples, rather than model training or fine-tuning.

Seven documented changes distinguish v2 from v1: separate endpoint requests from aisle traversals; infer from map descriptions before clarifying; define clockwise and counter-clockwise order explicitly; expand wall sweeps to corner-midpoint-corner; choose a named point for geometric descriptions; translate “wait at X” into navigation followed by waiting; and explicitly request `wait_retry` for retry instructions. The revisions target observed error categories. Because they followed inspection of evaluation failures and reuse the same dataset, their measured improvement is a development-set result, not held-out generalisation.

There is direct overlap between prompt examples and test commands: v2 includes the exact text of L1-01, L2-03, and

L4-09. General policies can still be useful, but example overlap prevents describing the comparison as free from test exposure. The baseline records are evaluated under the archived contract that includes `wait_retry`; the comparison does not establish an isolated historical schema change for v1 versus v2.

D. Simulation and motion execution

The robot is a TurtleBot3 Waffle in Gazebo Classic 11 under ROS2 Humble in WSL2. A script generates the occupancy grid and warehouse geometry from common dimensions. Grid resolution is 0.05 m, giving a 400×400 map. Aisle centre lines lie at $x = 6, 9, \text{ and } 12$ m. All 20 named locations were checked against the stored grid and occupy free cells; this check does not by itself certify footprint clearance or feasible routes.

Localisation for the demonstrated configuration uses simulator ground-truth odometry, a map server, and a static identity transform from `map` to `odom`. SLAM Toolbox is not running in this path. The launch file also offers AMCL, but that alternative is not the basis of the reported result. Ground truth reduces localisation uncertainty, allowing the experiment to concentrate on plan translation and integration.

Nav2 uses NavFn for global planning and Regulated Pure Pursuit for path following in the stored configuration. Desired linear velocity is 0.26 m/s; configured goal tolerances are 0.25 m in position and 0.25 rad in yaw. The executor constructs map-frame pose goals and calls `BasicNavigator.goToPose`. Goal orientation is derived from the previous recorded position towards the next target; the language contract does not specify approach direction or final orientation.

The ROS-free `plan_runner.py` implements sequencing, waiting, and failure handling. Seven existing mock-based tests pass, covering simple navigation, clarification without execution, abort, skip, retry, perimeter rerouting, and waiting. These verify selected control paths, not complete JSON validation or physical navigation. The ROS adapter reads a file or stdin and logs outcomes; it does not subscribe to a plan topic or publish a dedicated plan-status topic.

E. Engineering observations and demonstration

Development notes record communication problems with Fast DDS under WSL2 and a working configuration using Cyclone DDS restricted to loopback. They also record action-handshake timeouts in an uncomposed Nav2 launch and successful operation after composition. The checked-in launch hosts the navigation stack in a component container. These observations justify the chosen configuration, but no controlled middleware benchmark establishes a unique root cause or general performance improvement.

The archived demonstration requests: “Sweep aisle 1, check the packing station and the loading dock, then return to base.” Its five goals are `aisle_1_south`, `aisle_1_north`, `packing_station`, `loading_dock`, and `charging_dock`. The saved replay reports 5/5 goals reached, with a corresponding trajectory. This is the route supported by the stored artifact; it differs from the illustrative two-aisle patrol used elsewhere in development notes.

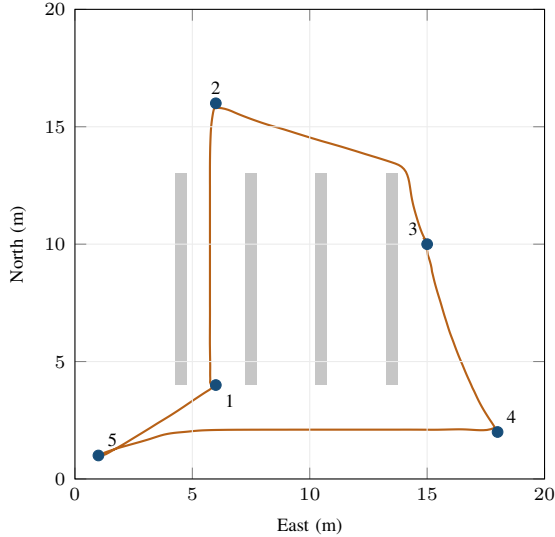


Fig. 2. Warehouse shelf geometry and archived demo trajectory. The numbered goals are aisle 1 south (1), aisle 1 north (2), packing station (3), loading dock (4), and charging dock (5). The plotted trace is stored odometry, not a newly executed experiment.

This demonstration establishes multi-goal integration, not a mission-success probability, physical inspection, or conditional behaviour. It is separate from the plan-level evaluation.

Follow-up checks on 12 September replayed three saved v2 outputs (trial 0): L1-01 reached 1/1 goals, L2-01 reached 3/3, and L3-01 reached 5/5. Each used a fresh headless world and the unchanged executor. Recorded odometry corroborated all nine destinations within the configured 0.25 m tolerance. An initial attempt lacked a resolved ground-plane model, followed by a startup timeout; both are retained as environment failures. The successful replays followed model-path and process-cleanup repairs. These selected single runs add execution evidence, not independent language samples or conditional-recovery validation.

IV. EVALUATION METHOD

A. Dataset and generation protocol

The dataset contains 20 commands per category (Table III). Items are authored around a fixed map and expected robot capabilities. L1–L3 use accepted destination sequences or required-visit rules; L4 includes both automatically checkable contingencies and rubric cases; L5 combines resolvable defaults, missing information, nonexistent locations, and capability limits. Thus, the categories vary in more than sentence complexity.

The main v1 and v2 runs were saved on 1 August 2026. Each contains three generation trials for all 100 commands, giving 300 records per prompt version. The v3 run, saved on 7 August, repeats only the 20 L4 commands, producing 60 records. All use the recorded model identifier gpt-4o-mini and temperature zero. These are repeated outputs for the same commands, not additional independent command samples. The record identifies the model alias rather than an immutable model snapshot, limiting exact future reproducibility.

TABLE III
COMMAND CATEGORIES AND DATASET EXAMPLES

Level	Category	Example
L1	Direct	Go to the loading dock.
L2	Spatial	Move along the east wall from south to north.
L3	Multi-step	Patrol aisles 1 and 3, then return to the charging dock.
L4	Conditional	Inspect storage zone A; if unreachable, inspect zone B instead.
L5	Ambiguous	Go check that thing near the door.

Each record stores the command ID, trial, raw response, latency, parse status, schema and map errors, and scoring fields. No new model calls were made for this report revision. The analysis uses the archived files and grades; it does not replace missing manual judgements with inferred passes.

B. Four evaluation measures

Parse validity asks whether the response can be decoded as JSON. *Schema adherence* asks whether the parsed object passes the chosen schema. *Map validity* asks whether navigation targets exist in the semantic map. *Semantic correctness* asks whether the represented task matches the instruction under the dataset’s matching rules or rubric. The checks are conceptually distinct but operationally sequential: schema rejection prevents later scoring in the runner.

For automatic semantic scoring, sequence items compare ordered navigation-target lists against accepted alternatives. Required-visit items check destination membership and, where specified, order, forbidden locations, or the existence of a wait action. Contingency expectations are recorded separately. These checks are reproducible but incomplete proxies for behaviour: they do not generally validate wait durations, physical route geometry, or every action relationship. A destination-list match should not be interpreted as complete executable equivalence.

The runner checks map membership only after schema success. Consequently, an unpopulated `map_error` on a schema-invalid record is not evidence of map validity. Reported map-check counts include only responses that actually reached that stage. For v3, the unchanged map checker inspects ordinary navigation targets but does not validate `fallback_target`; the narrow branch diagnostic checks selected expected fallback names separately.

C. Manual grading and denominators

There are 28 rubric-designated commands: 11 at L4 and 17 at L5. L4-08 fails schema validation in every v2 trial and does not enter manual semantic grading. The archived grading sheet therefore contains 27 items: 10 L4 and 17 L5. Their displayed outputs correspond to the first stored generation, apart from a punctuation-encoding difference in L5-18. Each was graded once by one researcher.

The recorded categories are pass, partial, and fail. The strict judgement reserves a pass for an acceptable represented

response; partial credit records cases with some intended content but incorrect or incomplete executable behaviour. In particular, naming an alternative destination in comments or later unconditional steps can satisfy a permissive rubric description yet receive only partial credit under the stricter behavioural interpretation. This distinction must remain visible because it affects the central finding.

Development notes describe a blind, shuffled procedure with seed 4445. However, the archived v2 grading interface identifies v2 in its heading and displays items in ID order. The stored artifact therefore does not establish effective blinding or that shuffle. This report treats the labels as single-researcher judgements, without asserting independent blinded assessment or inter-rater agreement [17].

For L1–L3, strict rates use 60 generation records per level. For L4–L5, the descriptive summary combines the stable automatic item outcomes with the once-graded rubric items, giving 20 item outcomes per level. Specifically, L4 has nine automatic passes plus two manual passes, six partials, and three failures; L5 has three automatic passes plus 14 manual passes and three partials. The L4 item summary does not incorporate the later schema-invalid generations of L4-17 and L4-19 as additional manual failures.

For N outcomes, P passes, and H partials, the reported scores are

$$S_{\text{strict}} = \frac{P}{N}, \quad S_{\text{weighted}} = \frac{P + 0.5H}{N}. \quad (1)$$

The weighted score is partial credit, not the proportion of fully successful tasks. No pooled success rate is computed across the mixed denominators. The L4–L5 comparison additionally reports descriptive Wilson intervals and a two-sided Fisher exact test on pass versus non-pass, with the qualifications in Section VI.

D. Targeted fallback intervention

Version v3 adds `goto_fallback` and `fallback_target` to the contract. The associated prompt policy explicitly instructs the model to place the alternative on the failed step and not emit it as a later unconditional navigation. The schema requires a nonempty fallback string when the new action is selected. This is one conceptual feature implemented through coordinated schema and prompt changes, not a schema-only ablation.

The structural comparison tests L4 schema adherence and a diagnostic subset of three commands: L4-12, L4-13, and L4-17. A branch passes that diagnostic when the first emitted fallback has the specified destination and that destination is not repeated in subsequent unconditional navigation targets. The check is intentionally narrower than complete semantics: it does not establish correct trigger attachment, traversal geometry, recovery after fallback failure, or executable robot behaviour. The v2 baseline is zero by construction because the field does not exist. The archived executor has no `goto_fallback` implementation, so v3 is evaluated as a representation experiment only.

TABLE IV
STRUCTURAL CHECKS ON V2 GENERATION RECORDS

Level	JSON parses	Schema passes	Map passes/checks
L1	60/60	60/60	60/60
L2	60/60	60/60	60/60
L3	60/60	60/60	60/60
L4	60/60	55/60	55/55
L5	60/60	60/60	60/60
Total	300/300	295/300	295/295

TABLE V
AUTOMATIC SEMANTIC RESULTS FOR THE PROMPT COMPARISON

Level	Passing records / 60		Mean passes / 20	
	v1	v2	v1	v2
L1	54	58	18.0	19.3
L2	48	57	16.0	19.0
L3	51	57	17.0	19.0

V. RESULTS AND ANALYSIS

A. Structural validity

All 300 v2 responses parse as JSON. L1, L2, L3, and L5 each have 60/60 schema-valid responses. L4 has 55/60, giving 295/300 overall. All 295 responses reaching the ordinary-target map check pass it. Table IV separates the checks to avoid assigning unperformed checks a success.

The five rejected L4 responses contain eight out-of-enum `on_blocked` fields using three distinct values. L4-08 emits `wait` twice in each of its three responses (six fields). L4-17 emits `navigate` once and L4-19 emits `try_aisle_1_north` once. Thus, “three values,” “five responses,” and “eight fields” describe different counts. The last value is particularly suggestive of an attempted alternative destination. The `wait` errors also show that not every violation is explained by the absence of destination fallback: a retry behaviour already exists.

B. Prompt revision and repeatability

On L1–L3, v1 obtains 54/60, 48/60, and 51/60 automatic semantic passes; v2 obtains 58/60, 57/60, and 57/60. The combined result is 153/180 (85.0%) versus 172/180 (95.6%), an increase of 19 passing records or 10.6 percentage points. Table V also gives the three-trial mean in passes per 20 commands.

Seven commands improve on L1–L3 and none regress. The often-quoted total of eight improved commands includes L5-13, outside that three-level comparison. Because revisions were developed after inspecting failures and some examples overlap with test items, these gains demonstrate improvement on this command set rather than unbiased performance on unseen requests.

Across all five categories, the recorded scoring outcomes remain identical over trials for 99/100 commands under v1 and 97/100 under v2. The three varying v2 cases are L1-18, L4-17, and L4-19. This is score repeatability, not output determinism: comparing full parsed JSON objects, including comments,

TABLE VI
SEMANTIC SUMMARY WITH EXPLICIT SCORING UNITS

Level	Scoring unit	Strict	Weighted
L1	60 records; 58 pass	96.7%	96.7%
L2	60 records; 57 pass	95.0%	95.0%
L3	60 records; 57 pass	95.0%	95.0%
L4	20 items; 11 pass	55.0%	70.0%
L5	20 items; 17 pass	85.0%	92.5%

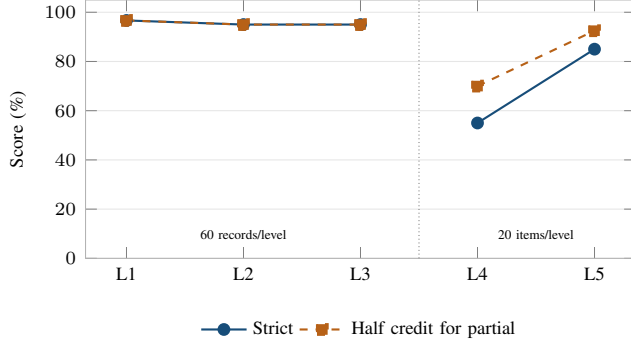


Fig. 3. v2 semantic scores. L1–L3 use repeated automatic records; L4–L5 combine automatic item outcomes with once-graded rubric outputs. The separation marks the change in scoring unit. Connecting segments guide the eye and do not imply a calibrated difficulty scale.

yields identical objects for only 43/100 v1 commands and 50/100 v2 commands. Stable scores can conceal different explanations or plans that share the same score. Temperature zero therefore does not justify treating the output as invariant.

C. Category scores and the L4–L5 contrast

Table VI and Fig. 3 show the descriptive pattern. Strict rates on L1–L3 are 96.7%, 95.0%, and 95.0%. L4 has 11 passes, six partials, and three failures, giving 55.0% strict and 70.0% weighted. L5 has 17 passes and three partials, giving 85.0% strict and 92.5% weighted. L5 exceeds L4 but remains below L1–L3.

The 30-percentage-point L4–L5 difference is descriptive. With 20 items per category, 95% Wilson intervals are 34.2–74.2% for L4 and 64.0–94.8% for L5. A two-sided Fisher exact test on [11, 9] versus [17, 3] gives $p = 0.0824$, not below 0.05. These calculations neither prove equality nor establish a population-level reversal. The authored categories and unequal success criteria further limit inference. The stronger argument concerns identifiable encoding failures and the targeted intervention.

D. Clarification and missing conditional structure

In the first v2 generation, 11/20 L5 responses defer using `understood=false`. L5-18, “Go to aisle 5,” correctly asks which of the three existing aisles is intended. That is an appropriate response for this rubric, but it completes no navigation task. Clarification is not automatically correct for every ambiguous-looking instruction: map descriptions or an accepted default can make a request resolvable. L5 success

TABLE VII
DESCRIPTIVE TAXONOMY OF MANUALLY GRADED L4 ITEMS

Requested structure	IDs (L4-)	Grades
Retry the same destination	09, 16	2 pass
Alternative/follow-on task	02, 12, 13, 17, 19	5 partial
Aggregate condition	10, 15	partial, fail
Approach geometry	20	fail

measures an acceptable response policy, including deferral, rather than the rate of completed robot missions.

Alternative-destination conditionals expose a different requirement. L4-12 requests inspection of zone A, with zone B only if A is unreachable. The first v2 output places A and B in separate ordinary navigation steps. Its second step contains the reason “Navigate to storage zone B if storage zone A is unreachable.” The executor does not interpret this sentence. With successful navigation to A, it still proceeds to B, contradicting the requested condition.

The following projection retains the relevant executable fields of that saved response; explanatory fields are omitted for compactness.

```

"plan": [
  {
    "action": "navigate",
    "target": "storage_zone_a",
    "on_blocked": "reroute_perimeter"
  },
  {
    "action": "navigate",
    "target": "storage_zone_b",
    "on_blocked": null
  }
]

```

This output recognises the conditional wording in its explanation but does not preserve it in executable structure. That observation is consistent with a representation bottleneck. It cannot establish that the model’s comprehension is otherwise perfect, since the explanation itself is generated text and the prompt explicitly directs residual alternatives into notes. The observed behaviour may reflect both the restricted contract and the instructed workaround.

The ten manually graded L4 cases help localise the limitation (Table VII). Retry requests pass where a corresponding action exists. Six partials involve alternative or contingent follow-on behaviour, but they are not six instances of a simple single-target fallback. They include multi-waypoint tasks, aggregate conditions, and recovery after fallback failure. This distinction is necessary to interpret v3’s scope.

E. What v3 repairs, and what it does not

For the 60 L4 records, v3 raises schema adherence from 55/60 to 60/60. The eight out-of-enum field occurrences disappear. On the selected branch diagnostic, L4-12 and L4-13 pass in all three trials. L4-17 passes in one trial; in the other two it names the fallback but also visits it unconditionally. The resulting 7/9 is a structural encoding result over three selected commands, not a semantic success rate over L4.

The baseline diagnostic is zero because the old contract cannot express the field being tested. The informative observation is therefore how often the new representation is used without flattening the branch. Since v3 also changes the prompt policy,

TABLE VIII
L4 INTERVENTION: MEASURED STRUCTURAL OUTCOMES

Measure	v2	v3
Schema-valid responses	55/60	60/60
Invalid contingency-field occurrences	8	0
Selected branch diagnostic	0/9	7/9
L4 semantic pass rate	55% ^a	Ungraded

^aItem-level v2 summary; not 60 graded generations.

this result supports the combined intervention; it does not isolate the independent causal effect of schema syntax.

Several residual requirements exceed a single fallback destination. L4-02 needs an alternative multi-waypoint patrol. L4-10 and L4-15 require state aggregated over blocked aisles. L4-20 requires approach geometry. L4-19 additionally asks for termination if the alternative also fails; it lies outside the three-command diagnostic and still repeats its fallback unconditionally in all three v3 outputs. Together with the two L4-17 failures, this prevents claiming that every remaining error lies outside the feature’s intended scope.

The new field is neither executed nor completely validated by the current downstream stack. The executor has no handler for it, and the ordinary map validator does not inspect its target. Moreover, 33/60 v3 records remain marked manual. Schema adherence and branch placement are useful intermediate outcomes, but successful task execution requires the schema, semantic checks, and executor to share the same meaning. That integration and independent semantic grading remain unfinished.

F. Response latency

For the 300 v2 records, mean measured latency is 2.23 s, median 2.08 s, and the 95th percentile 3.63 s. The percentile uses Python’s exclusive quantile interpolation; an inclusive convention gives 3.56 s on the same observations. The timer begins before client construction and ends after the response is received, before JSON parsing and validation. These values describe observed planner-call latency, including local client and network overhead, rather than pure model inference time or robot mission duration. No real-time or deployment service-level guarantee is inferred.

VI. DISCUSSION AND THREATS TO VALIDITY

A. Interpretation and practical significance

The representational limitation is straightforward once identified. The empirical contribution locates it in saved plans, distinguishes it from parsing and map errors, and tests an intervention with a measurable boundary. A larger model may improve selection among representable plans, but cannot make an unimplemented field executable without changing the surrounding system.

The practical implication is to check whether the vocabulary expresses the required behaviour, the validator checks it, and the executor implements it. Here, v2 has valid but unconditional alternatives, while v3 emits branches its executor cannot run. A syntactic improvement alone does not establish a robotics improvement.

B. Unequal response policies and construct validity

The L4–L5 contrast has a scoring confound. L5 often passes by appropriately declining to act, while L4 generally requires a specific conditional plan. The representation provides a deferral mechanism but lacks some conditional mechanisms, so the asymmetry is both part of the proposed explanation and a reason not to interpret the score gap as a pure language-complexity effect. A stronger evaluation would report correct execution plans, appropriate deferrals, and inappropriate deferrals separately under predefined task requirements.

L1–L5 is a researcher-designed taxonomy, not a psychometrically or linguistically validated ordinal scale. Ambiguity, sequence length, spatial grounding, and conditional control are different dimensions. Success need not decrease monotonically with those labels, even for a better planner. The defensible claim is that representational support helps explain this dataset’s error distribution; the experiment does not show that linguistic complexity has no effect.

C. Researcher choices, grading, and statistical limits

One researcher designed the schema, dataset, prompts, expected plans, and manual judgements. Prompt revision used observed failures; three examples exactly match dataset items. There is no held-out test set, independent dataset designer, external baseline, or comparison across model sizes. The absolute 55% L4 score consequently has no external reference point, and the v1–v2 gain may reflect adaptation to the development set.

Manual grading uses only one output per rubric item, while other categories use three generations. The strict interpretation is more demanding than parts of the original rubric, and the saved interface does not substantiate blinding. A second rater, a frozen rubric, and agreement measurement would improve confidence [17]. Repeated generations should be analysed as clustered within commands; counting all three as independent task samples would exaggerate precision. The reported intervals and Fisher test are descriptive diagnostics on authored items, not design-based estimates for all warehouse language.

The v3 subset is selected around an anticipated feature benefit, and its zero baseline is definitional. The intervention changes prompt instructions as well as the schema and was run on a later date using a model alias. Without a schema-by-prompt ablation or immutable model snapshot, attribution remains limited. Complete v3 semantic grades are needed before claiming an improvement in task correctness.

D. Robotics and deployment limits

Navigation evidence comprises the archived demonstration and three selected replays, one run per replay after environment repair. Most outcomes remain plan judgements, not observed robot behaviour. The fixed map, ground-truth localisation, static warehouse, and simulated platform remove important sources of real-world uncertainty. No systematic experiments quantify obstacle-triggered branches, localisation drift, collisions, mission duration, or recovery success.

The implementation also lacks a common validation gate across evaluation and execution, complete cross-field schema

constraints, and v3 fallback execution. “Blocked” is inferred from goal failure, and named-waypoint matching cannot prove an exact patrol path or physical inspection. These limitations preclude a safety guarantee or a supported claim of operational deployment maturity. They do not invalidate the saved translation results, but they constrain what those results establish about a robot.

VII. CONCLUSION AND FUTURE WORK

On this development dataset, typed language translates into restricted navigation plans with automatic semantic scores near 95% for direct, spatial, and multi-step requests. Conditional commands score below ambiguous commands, but that descriptive difference is not statistically conclusive and does not isolate linguistic difficulty.

Saved failures identify representation as one limiting factor: a condition can survive in a comment while disappearing from executable control flow. A fallback field and corresponding instructions improve schema adherence and selected branch encoding. This supports a bounded diagnosis, not complete semantic correctness or successful conditional navigation.

The highest-priority next step is validation of v3: freeze the grading rules, grade all relevant outputs with a second rater, and report agreement and disagreements. Implementing and validating fallback execution should then precede obstacle-controlled simulation trials that distinguish primary success, fallback success, and failure of both. The ordinary schema and map checks should become a shared execution gate, with explicit cross-field constraints and validation of fallback targets.

Further experiments should use unseen commands and another independently designed map, compare against a simple baseline and additional models, and separate prompt changes from representation changes. Richer representations for multi-step fallbacks, aggregate conditions, or approach constraints can then be tested against defined requirements rather than added without evaluation. Hardware trials and voice input remain later extensions. For the present system, strengthening evidence about existing behaviour has higher value than increasing feature count.

ARTIFACT AVAILABILITY

Code, prompts, dataset, and archived evaluation records are associated with <https://github.com/twillur/PDE4445-LLM-Nav2-Planner>. The project journal is at <https://twillur.github.io/PDE4445-Robotics-Dissertation/>. The report’s numerical checks use the local archived August 2026 runs; the journal is contextual material, not a substitute for those records.

USE OF GENERATIVE AI

Generative AI assistance was used to draft and revise this report, inspect code and saved results, check numerical claims and references, and prepare the LaTeX document and figures. This included substantive prose generation and analysis support. Assistance also supported setup repairs and follow-up Gazebo execution of saved plans. No new LLM evaluation samples or manual grades were generated.

REFERENCES

- [1] L. Pallottino, “Robotics for warehouses and logistics: Technologies, challenges, and future directions,” *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, pp. 377–401, 2026. [Online]. Available: <https://doi.org/10.1146/annurev-control-032724-020213>
- [2] M. Ahn *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [3] W. Huang *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.05608>
- [4] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022. [Online]. Available: <https://arxiv.org/abs/2209.07753>
- [5] I. Singh *et al.*, “ProgPrompt: Generating situated robot task plans using large language models,” *arXiv preprint arXiv:2209.11302*, 2022. [Online]. Available: <https://arxiv.org/abs/2209.11302>
- [6] S. Vempala, R. Bonatti, A. Bucker, and A. Kapoor, “ChatGPT for robotics: Design principles and model abilities,” *arXiv preprint arXiv:2306.17582*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.17582>
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [8] D. Shah, B. Osinski, B. Ichter, and S. Levine, “LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action,” *arXiv preprint arXiv:2207.04429*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.04429>
- [9] G. Zhou, Y. Hong, and Q. Wu, “NavGPT: Explicit reasoning in vision-and-language navigation with large language models,” *arXiv preprint arXiv:2305.16986*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.16986>
- [10] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2003.00368>
- [11] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2,” *Robot. Auton. Syst.*, vol. 168, p. 104493, 2023. [Online]. Available: <https://arxiv.org/abs/2307.15236>
- [12] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” *J. Open Source Softw.*, vol. 6, no. 61, p. 2783, 2021. [Online]. Available: <https://doi.org/10.21105/joss.02783>
- [13] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [14] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-Y. Lee, and Y.-N. Chen, “Let me speak freely? A study on the impact of format restrictions on performance of large language models,” *arXiv preprint arXiv:2408.02442*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.02442>
- [15] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, “JSONSchemaBench: A rigorous benchmark of structured outputs for language models,” *arXiv preprint arXiv:2501.10868*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10868>
- [16] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: <https://doi.org/10.1145/3571730>
- [17] R. Artstein and M. Poesio, “Survey article: Inter-coder agreement for computational linguistics,” *Comput. Linguist.*, vol. 34, no. 4, pp. 555–596, 2008. [Online]. Available: <https://aclanthology.org/J08-4004/>